

Synchronous vs Asynchronous in JS

1. What is Synchronous Javascript?

- a. Code runs *line by line* in the order it is written.
- b. If one task takes time, the next task has to *wait*.
- c. This is called **blocking**.
- d. **For Example** → Imagine you're in a queue at a coffee shop, and the barista only takes the next customer's order after completing the current one. Everyone waits in line.

2. What is Asynchronous JavaScript?

- a. Code does *not wait* for a task to finish.
- b. Long tasks (like fetching data from a server) run in the background.
- c. JavaScript moves on to the next task and comes back later when the result is ready.
- d. This keeps the app **fast and responsive**.
- e. **For Example** → At the same coffee shop, you place your order, get a token, and sit down. The barista can take other orders while your coffee is being made. You're notified when your coffee is ready.

3. Why Do We Need Asynchronous JavaScript?

- a. Real apps often fetch data, read files, or call APIs — these take time.
- b. If JavaScript was synchronous, the app would **freeze** until the task finishes.

c. Asynchronous code makes apps:

- ✓ Faster
- ✓ Smoother
- ✓ User-friendly

👉 Real-Life Example:

- **Synchronous:** A weather app freezes until the data is fetched.
- **Asynchronous:** The app shows a loading spinner, stays usable, and updates when data arrives.

4. Ways to Do Asynchronous JavaScript

- a. **setTimeout / setInterval** → Run code after a delay or repeatedly.
- b. **Promises (ES6)** → Handle results of async tasks more cleanly.
- c. **async / await** → Easier way to write promises, looks like normal code.
- d. **Fetch API** → Modern way to get data from servers.
- e. **AJAX (XMLHttpRequest)** → Old way of fetching data from a server.

5. Tools and Libraries for Asynchronous JavaScript:

a. Axios:

- A library to make **HTTP requests** (getting/sending data from servers).
- Works really well with **async/await**.
👉 Example: Fetching weather data from an API.

b. Node.js:

- A runtime environment that allows you to run JavaScript outside the browser.
- Uses asynchronous programming to handle file reading, databases, and network requests efficiently.
- Perfect for building fast, scalable server-side applications.

👉 Example: A chat app where multiple users send messages at the same time.

c. Web Workers:

- Allow JavaScript code to run in a background thread (separate from the main UI).
- Prevents heavy tasks (like image processing or calculations) from freezing the page.

👉 Example: Editing a large image in the browser without making the page unresponsive.

6. Important Concepts in Async JavaScript:

- a. **Event Loop:** The event loop is the mechanism that allows JavaScript to handle asynchronous tasks. It manages the execution of code, callbacks, and events. It works with the call stack, callback queue, and Web APIs to ensure tasks are executed in the correct order.
- b. **Microtasks vs Macrotasks:**
 - *Microtasks* → Promise callbacks (run first).
 - *Macrotasks* → setTimeout, setInterval (run after microtasks).
- c. **Callbacks:** A function passed to another function to run later.
- d. **Web APIs:** Browser features (like fetch, setTimeout, DOM events) that work asynchronously.
- e. **Single-threaded but Non-blocking:** JS has one main thread, but async code prevents it from getting stuck.